



Programación II - TUPAD

Trabajo Práctico

Integrador —

Food Store

Alumno: Luciana Belén Hamade

DNI : 37983197

Fecha: 25/6/2026

Índice

1. Marco Teórico	3
2. Decisiones Técnicas y Arquitectura.....	4
2.1 Organización de paquetes	4
2.2 Capa de Entidades	4
2.3 Capa de Gestores	5
2.4 Capa de Interfaz de Usuario (UI)	5
2.5 Excepciones personalizadas	6
2.6 Patrones de relación aplicados	6
3. Dificultades y Soluciones	7
4. Bibliografía / Webgrafía	8
5. Links del Proyecto	8

1. Marco Teórico

El presente trabajo práctico fue desarrollado íntegramente en **Java (versión 25)**, un lenguaje de programación orientado a objetos, fuertemente tipado. Java permite estructurar el código en jerarquías de clases, lo que resulta fundamental para modelar entidades del mundo real y sus relaciones.

Conceptos aplicados en el proyecto:

• Programación Orientada a Objetos (POO)

Paradigma central del proyecto. Se aplicaron los cuatro pilares: abstracción (clase Base abstracta), encapsulamiento (atributos privados con getters/setters), herencia (todas las entidades extienden Base) y polimorfismo (toString() abstracto implementado de forma distinta en cada subclase).

• Herencia y Clases Abstractas

La clase Base centraliza atributos comunes a todas las entidades: id (Long autoincremental), eliminado (boolean para baja lógica) y createdAt (LocalDateTime). Al declarar toString() como abstracto, se obliga a cada subclase a implementarlo, garantizando que cualquier objeto del sistema sea imprimible de forma clara para cualquier usuario.

• Interfaces

La interfaz Calculable define el contrato calcularTotal(): void, implementado exclusivamente por la clase Pedido. Esto permite que el cálculo del total se invoque de forma polimórfica y se actualice automáticamente al modificar sus detalles.

• Enumeraciones (Enums)

Se utilizaron tres enums para representar valores de dominio cerrado: Rol (ADMIN, USUARIO), Estado (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO) y FormaPago (TARJETA, TRANSFERENCIA, EFECTIVO). Los enums garantizan que solo se asignen valores válidos.

• Excepciones personalizadas

Se implementaron excepciones propias extendiendo Exception para representar errores de negocio con semántica clara: EntidadNoEncontradaException, MailDuplicadoException, NombreDuplicadoException, PrecioInvalidoException, StockInvalidoException y ProductosAsociadosException.

• Colecciones

Se utilizaron listas ArrayList para gestionar colecciones de entidades en memoria, dado que el proyecto no utiliza base de datos. Cada Gestor mantiene su propia lista estática como repositorio durante la ejecución.

• Java Time API

Se usaron LocalDate para la fecha del pedido y LocalDateTime para el timestamp de creación (createdAt en Base), pertenecientes al paquete java.time introducido en Java 8.

2. Decisiones Técnicas y Arquitectura

2.1 Organización de paquetes

El proyecto adopta una arquitectura en capas, separando responsabilidades en seis paquetes distintos:

Paquete	Contenido	Responsabilidad
Entities	Base, Categoria, Producto, DetallePedido, Pedido, Usuario Modelo del dominio.	Atributos, relaciones y comportamiento de cada entidad
Enums	Rol, Estado, FormaPago	Valores de dominio cerrado. Garantizan que solo se asignen opciones válidas.
Interfaces	Calculable	Contratos de comportamiento. Calculable obliga a Pedido a implementar calcularTotal
Exceptions	6 excepciones personalizadas	Creadas para cumplir con las historias de usuario ,lanzadas desde los Gestores.Trabajan sobre errores de negocio
Gestores	GestorCategorias, GestorProductos, GestorUsuarios, GestorPedidos	Capa de servicio. Contienen la lógica de negocio y las listas en memoria.
Ui	Menu, MenuCategorias, MenuProductos, MenuUsuarios, MenuPedidos	Capa de presentación. Interacción con el usuario via consola.

2.2 Capa de Entidades

Todas las entidades heredan de **Base**, que centraliza id (generado con un contador estático autoincremental por clase), eliminado (permite baja lógica sin borrar registros) y createdAt (timestamp de creación). Base declara toString() abstracto, obligando a cada subclase a definir su propia representación textual.

Estructura de herencia:

Entidad	Extiende	Implementa	Atributos Clave
Base	—	—	id, eliminado, createdAt
Categoria	Base	—	nombre, descripcion, List<Producto>
Producto	Base	—	nombre, precio, stock, disponible, categoria

DetallePedido	Base	—	cantidad, subtotal, producto
Pedido	Base	Calculable	fecha, estado, total, formaPago, detalles, usuario
Usuario	Base	—	nombre, apellido, mail, contrasenia, rol, List<Pedido>

2.3 Capa de Gestores

Los Gestores actúan como capa de servicio. Cada uno mantiene una lista estática de su entidad correspondiente y concentra toda la lógica de negocio: validaciones, búsquedas, creación, modificación y baja lógica. La separación entre Gestor y Menú garantiza que la consola no contenga lógica de negocio.

GestorCategorias

CRUD completo. Validación de nombre duplicado (existeNombreCategoria()). Baja lógica impedida si la categoría tiene productos activos asociados (lanza **ProductosAsociadosException**, consultando a **GestorProductos.tieneProductosEnCategoria()**).

• GestorProductos

CRUD. Lanza **PrecioInvalidoException** si el precio es negativo y **StockInvalidoException** si stock es negativo. Incluye métodos especializados: **actualizarPrecio()**, **actualizarStock()**, **actualizarCategoria()** y **mostrarProductosPorCategoria()** para filtrar por categoría.

• GestorUsuarios

CRUD. Valida unicidad de mail en alta (existeMail()) y en edición (existeMail()) con overload que **excluye al propio usuario**). **modificarRolUsuario()** requiere que el operador sea **ADMIN**.

• GestorPedidos

Registra pedidos, los lista y aplica baja lógica. La baja también marca el estado como **CANCELADO** y no permite eliminar pedidos con estado **TERMINADO**.

2.4 Capa de Interfaz de Usuario (UI)

La capa UI está dividida en cinco clases. **Menu** actúa como lector de Scanner compartido: expone métodos **leerInt()**, **leerLong()**, **leerDouble()** y **leerTexto()** que son reutilizados por los submenús. Esto evita instanciar múltiples Scanner y centraliza el manejo de errores de entrada. Cada submenú (**MenuCategorias**, **MenuProductos**, **MenuUsuarios**, **MenuPedidos**) recibe el Menu por constructor (inyección de dependencia simple) y gestiona el ciclo do-while de su sección. Los try-catch están en la UI, que captura las excepciones lanzadas por los Gestores y las presenta al usuario con mensajes claros.

MenuPedidos incluye funcionalidad extendida: permite modificar un pedido existente con un submenú anidado que ofrece cambiar la forma de pago, agregar o eliminar productos del detalle, y actualizar el estado. El total se recalcula automáticamente en cada operación gracias a **calcularTotal()** en **Pedido**.

```

Accediendo a Gestión de Pedidos...

--- GESTIÓN DE PEDIDOS ---
1. Ver Historial de Pedidos
2. Hacer un Pedido
3. Cancelar un Pedido
4. Modificar un Pedido
0. Volver al Menú Principal
=====
Seleccione una opción (0-4): 2
ID del Usuario: 1
ID del Producto: 1
Cantidad: 3

--- SELECCIONE FORMA DE PAGO ---
1. EFECTIVO
2. TARJETA
3. TRANSFERENCIA
Seleccione opción (1-3): 1
Pedido registrado en el historial.

Pedido #1 creado [EFECTIVO]. Total: $10500.0

```

2.5 Excepciones personalizadas

Excepción	Cuándo se lanza
EntidadNoEncontradaException	buscarPorId() no encuentra la entidad activa con ese ID
MailDuplicadoException	crearUsuario() o editarUsuario() detecta mail ya registrado
NombreDuplicadoException	crearCategoria() detecta nombre ya existente (activo)
PrecioInvalidoException	CrearProducto() o actualizarPrecio() recibe precio negativo
StockInvalidoException	crearProducto() o actualizarStock() recibe stock negativo
ProductosAsociadosException eliminarCategoria()	detecta productos activos asociados

2.6 Patrones de relación aplicados

• Composición — Pedido → DetallePedido

Los **DetallePedido** se crean exclusivamente desde **addDetallePedido()** en **Pedido**. La lista detalles nace en el constructor con **new ArrayList<>()**. Nunca se instancia un **DetallePedido** desde afuera. Si el pedido se elimina lógicamente, los detalles pierden sentido junto con él.

• Agregación — Categoria → Producto

Los productos se crean de forma independiente en **GestorProductos** y luego se asocian a una categoría. Si la categoría se elimina lógicamente, los productos podrían reasignarse. La relación es bidireccional: **agregarProducto()** en **Categoria** también llama a **setCategoria(this)** en el **Producto**.

• Asociación — Usuario → Pedido y DetallePedido → Producto

Los pedidos se crean fuera del usuario y se le agregan mediante **agregarPedido()**, que también establece la referencia inversa (**setUsuario**). Los productos existen independientemente de los detalles y solo se referencian desde **DetallePedido**.

3. Dificultades y Soluciones

1. Gestión del ID autoincremental sin base de datos

Al no usar persistencia, fue necesario generar IDs únicos en memoria. La solución fue declarar un contador estático (`private static Long contadorId = 0L`) en cada entidad y usar `++contadorId` dentro del constructor al llamar a `super()`. Esto garantiza que cada instancia reciba un ID único durante la ejecución.

```
static Long contadorId = 0L;
public Entidad() { super(++contadorId); }
```

2. Evitar eliminar categorías con productos activos

Un requisito de integridad es que no se pueda eliminar una categoría que tenga productos activos. Se implementó el método `GestorProductos.tieneProductosEnCategoria(id)`, que `GestorCategorias` consulta antes de aplicar la baja lógica. Si existen productos, lanza `ProductosAsociadosException` con un mensaje descriptivo.

```
if (GestorProductos.tieneProductosEnCategoria(id)) {
    throw new ProductosAsociadosException(...);
}
```

3. Validación de mail único al editar usuarios

Al modificar el mail de un usuario, era necesario verificar que el nuevo mail no estuviera en uso por otro usuario, pero sí permitir que el usuario conserve su propio mail. Se resolvió con una sobrecarga de `existeMail(String mail, Long idRegistrado)` que excluye al propio usuario de la búsqueda de duplicados.

```
existeMail(nuevoMail, uMod.getId()); // excluye al mismo usuario
```

4. Separación limpia entre UI y lógica de negocio

Una dificultad de diseño fue evitar que los Menús contuvieran validaciones de negocio. La decisión fue que todos los controles (precio negativo, nombre duplicado, mail inválido) se lanzan como excepciones desde los Gestores. Los Menús solo capturan esas excepciones y las presentan al usuario, sin duplicar lógica.

```
// En Gestor:
if (precio < 0) throw new PrecioInvalidoException(...);
```

```
// En MenuUI:
try { GestorProductos.crearProducto(...); }
catch (PrecioInvalidoException e) { System.out.println(e.getMessage()); }
```

4. Bibliografía / Webgrafía

- **Documentación oficial de Java SE 25**
<https://docs.oracle.com/en/java/javase/25/>
- **Java Collections Framework — ArrayList**
<https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>
- **Java Enums — Tutorial oficial de Oracle**
<https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html>
- **Java Exceptions — Tutorial oficial de Oracle**
<https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- **Java Time API — LocalDate y LocalDateTime**
<https://docs.oracle.com/javase/8/docs/api/java/time/package-summary.html>

5. Links del Proyecto

Recurso	URL
Repositorio GitHub	https://github.com/LuchiHamade/FoodStore
Video explicativo	https://youtu.be/REmQceJZPbc